

SPOD BLE Interface v2.0

The sPOD mobile phone application communicates with the sPOD system over the BLE interface. This interface is in a state of flux, this document attempts to capture the current state with the understanding that the entire interface is subject to change. Document audience includes sPOD mobile application developers, sPOD firmware developers, sPOD firmware maintainers and whomever else feels compelled to read it.

The sPOD system consists of a number of components, all of which contain a BLE subsystem (i.e., BLE radio and communication stack). However, only PCM1 will enable the BLE interface. At least that is the current plan for the first firmware release. One of the advantages to this restriction is the operator only sees one sPOD BLE device when trying to connect to the sPOD system. The main disadvantage is PCM1 may be in a location where BLE coverage is not ideal so it would be nice to have the option to join one of the other devices in the sPOD system..

A BLE interface can be described by the device name, services and characteristics it offers.

The sPOD device name is: “**sPOD Link #AABBCC**” where AABBCC are the least significant 24 bits of PCM1’s MAC address. (Note: the MAC address is a 48-bit unique number where the most significant 24 bits are the BLE chip manufacturing company code and the least significant 24 bits are unique to that company (or device type).)

BLE services and characteristics are identified over the BLE interface using UUIDs (Universally Unique Identifiers, which are 128 bits and generated by a UUID generator). Note, these are embedded in the firmware and immutable. That is, the sPOD mobile application can rely on the advertised BLE services and BLE characteristic’s UUIDs to remain constant.

Following is a list of the UUIDs advertised by the sPOD BLE interface:

- **sPOD service:** { 0x5Bu, 0x68u, 0x56u, 0x60u, 0x9Fu, 0xE9u, 0x04u, 0x84u, 0x7Bu, 0x44u, 0x0Du, 0x8Cu, 0xECu, 0xF9u, 0x3Au, 0x7Eu }
- **Passkey characteristic:** { 0x86u, 0x74u, 0x26u, 0x03u, 0xEEu, 0x73u, 0x28u, 0x8Eu, 0xB7u, 0x45u, 0x5Cu, 0xF1u, 0xFBu, 0x52u, 0x9Eu, 0x1Bu }
- **COMM characteristic:** { 0xF4u, 0xC6u, 0x6Fu, 0x64u, 0x02u, 0xECu, 0x5Bu, 0xA6u, 0x6Fu, 0x4Cu, 0x00u, 0x4Cu, 0x64u, 0x47u, 0x06u, 0xB9u }
- **PKT Data characteristic:** { 0xA7u, 0x4Au, 0xA1u, 0xBCu, 0x1Fu, 0xA9u, 0x5Cu, 0xA3u, 0x66u, 0x48u, 0xA4u, 0x76u, 0x0Bu, 0x3Bu, 0x24u, 0x53u }
- **OTA Control characteristic:** { 0x75u, 0x16u, 0x74u, 0x8Eu, 0x97u, 0x60u, 0xE6u, 0xA6u, 0x9Eu, 0x43u, 0x93u, 0xFEu, 0xA5u, 0x93u, 0x89u, 0xDFu }
- **OTA Data characteristic:** { 0x55u, 0xA3u, 0xe0u, 0x28u, 0x19u, 0x5Cu, 0x69u, 0x90u, 0x58u, 0x4Cu, 0xAEu, 0xFEu, 0xA9u, 0x82u, 0x47u, 0xF9u }

1.0 sPOD Service

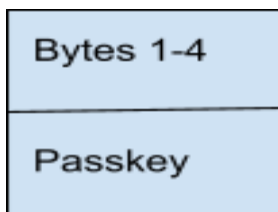
The PCM has a pairing button, whenever the operator wishes to pair a new mobile device to the PCM they must press the PCM pairing button and hold it down for at least 5 seconds. Once released the PCM will forget any previous device that was paired (i.e. the BLE bonding information will be deleted).

After the pairing button has been released, the new mobile device has 30 seconds to join PCM's BLE interface. Once joined it needs to write a 0x00 to the Passkey Characteristic. PCM will respond by generating a new unique key and writing it to the Passkey Characteristic. The mobile device will use the passkey to communicate with the other PCM encrypted characteristics.

The PCM will record this passkey and the MAC address of the mobile app that joined in flash (BLE bonding information). This mobile app can join the PCM in subsequent connections and will be remembered until the pairing key is pressed again, held down for 5 seconds and released, erasing this BLE bonding information.

2.0 Passkey Characteristic

The **Passkey** characteristic is not encrypted or authenticated (i.e., data is in the clear). This characteristic is only relevant after the PCM pairing button has been held down for 5 seconds and released. After the mobile app writes a 0 to the Passkey characteristic, the PCM writes the current passkey to this characteristic. If the mobile app does not write to the passkey characteristic within 30 seconds after the pairing button is released the window will close and PCM will no longer respond to mobile application writes to the Passkey characteristic.



3.0 COMM Characteristic

The **COMM** characteristic is used by PCM1 to send encrypted information to the mobile application. The only packet supported by v2.0 is the BOARD_INFO_PACKET (9):

- Length includes PacketType, Payload and CRC, so the length of the payload is (Length-5)
- PacketType is BOARD_INFO_PACKET
- Payload is the actual data
- CRC is the 32-bit CRC in little endian format

Byte 1	Byte 2	Byte 3	Bytes 4 .. N	Bytes N+1 .. N+4
0x55	Length	Packet Type	Payload	CRC

3.1 BOARD_INFO Payload format (new)

Following is the BOARD_INFO packet (including the COMM characteristic packet wrapper because I thought it was clearer):

Byte 1	Byte 2	Byte 3	Bytes 4..51	Bytes 52..55
0x55	Length (53)	Packet Type (9)	board_info[16]	CRC

Where board_info[16] is an array of 16 board_info_t elements. board_Info_t is a C Language structure:

```
typedef struct {  
    uint8_t mac[3];  
    uint8_t board_type;  
    uint8_t addr;  
    uint8_t major;  
    uint8_t minor;  
    uint8_t revision;  
} board_info_t;
```

whose fields are described below:

- uint8_t mac[3] – least significant 24-bits of board's BLE MAC address
- uint8_t board_type – the board type, where board types are:
0x00 – unspecified
0x10 – PCM
0x20 – SWITCH
0x30 – TOUCHSCREEN
- uint8_t addr – this is the RS485 local address (0x80-0x83 reserved for PCMx, SWITCH and TOUCHSCREEN boards have addresses starting at 0x84)
0x80 – PCM1
0x81 – PCM2
0x82 – PCM3
0x83 – PCM4
- uint8_t major – the major firmware version number
- uint8_t minor – the minor firmware version number
- uint8_t revision – the firmware revision

4.0 PKT Data Characteristic

The **PKT Data** characteristic is used by the sPOD mobile application and PCM1 to send and receive encrypted information. This is basically the same format used over the RS485 bus to send/receive data.

4.1 General PKT Format

Following is the general format of packets (each column is one byte, except for payload):

S T X	S E Q	T Y P E	M A C 1	M A C 2	M A C 3	L E N G	~ L E N G	PAYLOAD	C R C 1	C R C 2	E T X
2								0-255 bytes			3

STX – 0x02

SEQ – 8-bit sequence number (incremented for every packet sent)

TYPE – top 4 bits are board_type_t bottom 4 are pkt_type_t

MAC1,2,3 – 3-byte destination MAC address (bottom 24-bits of BLE MAC), all 1s for broadcast

LENG – number of bytes of data, does not include STX, SEQ, TYPE, MAC, LENG, CRC or ETX

~LENG – used to verify LENG byte, this is the INVERSE of LENG byte

PAYLOAD – LENG bytes of payload data, where LENG can be 0 to 255

CRC – 16 bit CRC with polynomial $x^{16}+x^{12}+x^5+1$ (crc of STX through PAYLOAD data)

ETX – 0x03

```
typedef enum {  
    UNSPECIFIED_BOARD_TYPE = 0x00,  
    PDB_BOARD_TYPE          = 0x10, // PCM  
    SPB_BOARD_TYPE          = 0x20, // SWITCH  
    TSB_BOARD_TYPE          = 0x30, // TOUCHSCREEN  
} board_type_t;
```

```
typedef enum {  
    HEARTBEAT_PKT_TYPE      = 0x00,  
    SYS_PKT_TYPE             = 0x01,  
    CONFIG_PKT_TYPE          = 0x02,  
    PCM_CONFIG_PKT_TYPE      = 0x03,  
    PCM_STATUS_PKT_TYPE      = 0x04,  
    SWITCH_CONFIG_PKT_TYPE   = 0x05,  
    SWITCH_STATUS_PKT_TYPE   = 0x06,  
    OTA_CTRL_PKT_TYPE        = 0x08,  
    OTA_CTRL_ACK_PKT_TYPE    = 0x09,  
    OTA_DATA_PKT_TYPE        = 0x0A,  
    RS485_PKT_TYPE           = 0x0B,  
    DEBUG_PKT_TYPE           = 0x0D,  
    CIRCUIT_STATUS_PKT_TYPE  = 0x0E,  
    MASK_PKT_TYPE            = 0x0F  
} pkt_type_t;
```

The Type field can also have the uppermost bit set PKT_SRCMAC (0x80) which signifies the source MAC address is prepended to the packet payload (and included in the LENG field).

4.2 Heartbeat Packet (0x00)

Each Baja device attached to the RS485 bus broadcasts a heartbeat packet every 5 seconds. The PAYLOAD section of the PKT is described by this C Language data structure:

```
typedef struct {  
    uint8_t addr;           // PCM1=0x80, PCM2=0x81, PCM3=0x82, PCM4=0x83, C1=0x84  
    uint8_t major;          // major version number  
    uint8_t minor;          // minor version number  
    uint8_t revision;       // revision  
    uint32_t beat;          // number of 5 second beats since boot  
} pkt_heartbeat_t;
```

The board type and source MAC address are sent as part of the packet meta (control) data. PKT_SRCMAC (0x80) is set in the TYPE field which means the source MAC address is included as part of the PAYLOAD data (and accounted for in the LENG field). Remember the top 4 bits of the TYPE field is the board type.

Since a Baja device sends the heartbeat every 5 seconds, device uptime is the 'beat' field times 5 seconds.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0x90	3 byte dest MAC			3 byte MAC (24 bits)	8 bytes (pkt_heartbeat_t)			3

TYPE – 0x90 bottom 4 bits are HEARTBEAT (0) and top 4 are 0x80 (SRCMAC) and 0x10 PCM

DSTMAC – 0xFFFFFFFF which denotes a broadcast packet

LENG - 0x0B (11 bytes = 3 + 8)

~LENG - 0xF4

4.3 Config Packet (0x02)

All of the config data (config_t, pcm_config_t and switch_config_t) are stored in flash and RAM. There are three formats for Config packets and they are basically determined by their respective lengths:

1. SYNC or UPDATE, the length is sizeof(config_t) (224 bytes)
2. PUT_CRCS this is 203 bytes long
3. GET_CRCS, APPLY, RESTORE, and GET (1 byte)

```
#define CONFIG_MAGIC 0xCAFE
```

```
#define CONFIG_VERSION 2
```

```
#define CONFIG_SIZEOF 224
```

```
#define CONFIG_APPLY_CMD      0
```

```
#define CONFIG_RESTORE_CMD    1
```

```
#define CONFIG_UPDATE_CMD     2
```

```
#define CONFIG_GET_CMD        3
```

```
#define CONFIG_SYNC_CMD       4
```

```
#define CONFIG_FACTORY_DEFAULTS_CMD 5
```

```
#define CONFIG_GET_CRCS_CMD    6
```

```
#define CONFIG_PUT_CRCS_CMD     7
```



```

typedef struct {
    uint16_t crc;                // CRC15 of remaining structure
    uint16_t magic;              // CONFIG_MAGIC
    uint16_t version;            // CONFIG_VERSION
    uint16_t sizeof_this;        // sizeof(config_t)
    // don't add anything before this

    uint32_t backlight_color;
    uint8_t backlight-brightness;
    uint8_t cfgindex;
    bool backlight;
    bool password_required;
    bool ignition_required;
    bool trigger_high;
    bool lockout;
    bool security;
    uint8_t password[32];
    uint32_t inactivity_timeout;
    uint32_t deepsleep_time;
    uint8_t circuit_min_amps;
    uint8_t circuit_max_amps;
    uint16_t shutoff_temp; // in tenths celsius, default is 1250 (125C)
    uint32_t avail_flags:31;
    uint32_t detect_opens:1;
    uint32_t power_scale;
    // 72 bytes to here

    // add new stuff here by taking away from future_growth[]
    uint8_t future_growth[152];
    // CONFIG_SIZEOF bytes to here
} config_t;

```

First format is for CONFIG_SYNC_CMD and CONFIG_UPDATE_CMD where the PAYLOAD is the config_t structure and the in-memory config is updated without writing to the file system.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xA2	3 byte dest MAC			3 byte MAC (24 bits)	224 bytes (config_t)			3

TYPE – 0xA2 bottom 4 bits are CONFIG_PKT_TYPE(2) and top 4 are 0x80 (SRCMAC) and 0x20 SWITCH

LENG - 0xE3 (227 bytes = 3 + 224)

~LENG - 0x1C

4.3.2 Config – APPLY Packet

The APPLY packet PAYLOAD is 1 byte and is set to CONFIG_APPLY_CMD(0). This will cause the device to write the in-memory config structure to the flash file named “config.bin”.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB2	3 byte dest MAC			3 byte MAC (24 bits)	1 bytes (0x00)			3

TYPE – 0xB2 bottom 4 bits are CONFIG_PKT_TYPE(2) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x04 (4 bytes = 3 + 1)

~LENG - 0xFB

4.3.3 Config – Restore Packet

The RESTORE packet PAYLOAD is 1 byte and is set to CONFIG_RESTORE_CMD(1). This will cause the device to read the flash “config.bin” file into the in-memory config structure.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB2	3 byte dest MAC			3 byte MAC (24 bits)	1 bytes (0x01)			3

TYPE – 0xB2 bottom 4 bits are CONFIG_PKT_TYPE(2) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x04 (4 bytes = 3 + 1)

~LENG - 0xFB

4.3.4. Config – Get Packet

The GET PAYLOAD is 1 byte and is set to CONFIG_GET_CMD(3). When sent to the PCM1 it will respond with a CONFIG_SYNC_CMD(4) (config_t payload) which causes the sending device to update the contents of the in-memory config structure.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB2	3 byte dest MAC			3 byte MAC (24 bits)	1 bytes (0x03)			3

TYPE – 0xB2 bottom 4 bits are CONFIG_PKT_TYPE(2) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x04 (4 bytes = 3 + 1)

~LENG - 0xFB

4.4 PCM Config Packet

There are basically two formats for PCM config packets, two-byte which are the PCM_APPLY_CMD, PCM_RESTORE_CMD, and PCM_GET_CMD packets and the PCM_UPDATE_CMD and PCM_SYNC_CMD where the payload includes the pcm_config_t structure.

```
#define PCM_MAGIC 0xCAFE
```

```
#define PCM_VERSION 2
```

```
// command for PCM_CONFIG_PKT_TYPE (if length < sizeof(pcm_config_t)
```

```
typedef enum {
```

```
    PCM_APPLY_CMD      = 0,
```

```
    PCM_RESTORE_CMD    = 1,
```

```
    PCM_UPDATE_CMD     = 2,
```

```
    PCM_GET_CMD        = 3,
```

```
    PCM_SYNC_CMD       = 4
```

```
} pcm_config_command_t;
```

```
#define TRIGGER_LATCHED 0x40
```

```
#define TRIGGER_ACTIVE_LOW 0x80
```

```
typedef struct {
```

```
    uint16_t crc;                // CRC16 of remaining pcm_config_t structure
```

```
    uint16_t magic;              // PCM_MAGIC
```

```
    uint16_t version;            // PCM_VERSION
```

```
    uint16_t sizeof_this;        // sizeof(pcm_config_t)
```

```
    // don't add anything before this
```

```
    uint16_t avail;
```

```
    uint8_t pcm_index;           // 0..3
```

```
    uint8_t circuit_count;       // either CIRCUITS/2 (12) or CIRCUITS (24)
```

```
    circuit_config_t circuits[CIRCUITS];
```

```
    uint8_t triggers[TRIGGERS];  // TRIGGER_LATCHED...
```

```
} pcm_config_t;
```

4.4.1 PCM Config – SYNC or UPDATE Packet

The SYNC or UPDATE PAYLOAD is the pcm_config_t structure and the in-memory config is updated without writing to the file system.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xA3	3 byte dest MAC			3 byte MAC (24 bits)	212 bytes (pcm_config_t)			3

TYPE – 0xA3 bottom 4 bits are PCM_CONFIG_PKT_TYPE(3) and top 4 are 0x80 (SRCMAC) and 0x20 SWITCH

LENG - 0xD7 (215 bytes = 3 + 212)

~LENG - 0x28

There is a maximum of 24 circuits and 8 triggers per PCM. The triggers can be active high/low and latched or momentary. Following is the circuit_config_t format:

```
typedef struct {
    uint16_t index;           // PCMS*CIRCUITS (4*24=96) (0..95)
    uint8_t output;           // circuit_output_type_t – OUTPUT_UNUSED ...
    uint8_t dim_level;        // 0-100% circuit PWM (starting DIM level)
    uint8_t alwaysOn:1;       // 1-bit – output always on (used to turn outputs back on
                               // when returning from deep sleep mode)
    uint8_t isIgnCtrl:1;      // 1-bit – only turn output if ignSense
    uint8_t isLockout:1;      // 1-bit – if true don't modify output
    uint8_t isCurrentRestart:1; // 1-bit – if true output is auto re-enable after current
                               // subsides when SHORT was detected

    uint8_t avail2:4;
    uint8_t currentLimit;     // specify the current limit 2A to 40A
    uint8_t strobeOn;         // milliseconds ON during strobing (STROBE/WIG/WAG)
                               // this period is PWMed with dim_level, units are 20ms
    uint8_t strobeOff;        // milliseconds OFF during strobing, this is *ALL OFF*
                               // units are in 20ms, range is 20ms to 5.1 seconds
} circuit_config_t;

typedef enum {
    OUTPUT_UNUSED = 0,
    OUTPUT_TOGGLE = 1,
    OUTPUT_STROBE = 2,
    OUTPUT_WIG = 3,
    OUTPUT_WAG = 4,
    OUTPUT_PRIMARY_MASK = 0x07,
    OUTPUT_SECONDARY_MASK = 0x70,
    PRIMARY_MOMENTARY = 0x08,
    SECONDARY_DOUBLE = 0x80
} circuit_output_type_t;
```

4.4.2 PCM Config – APPLY Packet

The APPLY PAYLOAD is 2 bytes and is set to PCM_CONFIG_APPLY_CMD(0). This will cause the device to write the in-memory pcm_config structure to the file named pcmN.bin. Where N is 0-3.

S T X	S T E Q U E	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB3	3 byte dest MAC			3 byte MAC (24 bits)	2 bytes (0x00, N)			3

TYPE – 0xB3 bottom 4 bits are PCM_CONFIG_PKT_TYPE(3) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x05 (4 bytes = 3 + 2)

~LENG - 0xFA

PAYLOAD – first byte is the PCM_CONFIG_APPLY_CMD(0), second byte is the PCM (0-3)

4.4.3. PCM Config – Restore Packet

The RESTORE PAYLOAD is 2 bytes and is set to PCM_CONFIG_RESTORE_CMD(1). This will cause the device to read the pcmN.bin file and store the contents in the in-memory pcm_config structure.

S T X	S T E Q U E	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB3	3 byte dest MAC			3 byte MAC (24 bits)	2 bytes (0x01, N)			3

TYPE – 0xB3 bottom 4 bits are PCM_CONFIG_PKT_TYPE(3) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x05 (5 bytes = 3 + 2)

~LENG - 0xFA

PAYLOAD – first byte is the PCM_CONFIG_RESTORE_CMD(1), second byte is the PCM (0-3)

4.4.4 PCM Config – Get Packet

The GET PAYLOAD is 2 bytes and is set to PCM_GET_CMD(3). This will be sent to the PCM1 which will respond with a PCM_SYNC_CMD (pcm_config payload) which causes the sending device to update the contents of it's corresponding in-memory pcm_config structure.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB3	3 byte dest MAC			3 byte MAC (24 bits)	2 bytes (0x03, N)			3

TYPE – 0xB3 bottom 4 bits are PCM_CONFIG_PKT_TYPE(3) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x05 (4 bytes = 3 + 2)

~LENG - 0xFA

PAYLOAD – first byte is the PCM_CONFIG_GET_CMD(3), second byte is the PCM (0-3)

4.5 PCM Status Packet

The PCM_STATUS_PKT_TYPE(4) payload is the pcm_status_t structure..

```
typedef struct {
    uint8_t witch:3;          // which PCM 0-3
    uint8_t ignSense:1;       // non-zero if ignition is sensed
    uint8_t tempEdge:1;       // non-zero if edge temperature of PCM board is okay
    uint8_t is13v_not24:1;    // non-zero if 12v battery, otherwise 24v
    uint8_t overheated:1;     // non-zero if temperature sensor > 125C
    uint8_t overheated2:1;    // non-zero if edge temperature sensor detected overheat
    uint8_t inputs;           // bitmap of the 8 triggers (raw input)
    uint8_t changed;          // bitmap of changed input bits from last status
    uint8_t triggers;         // bitmap of active triggers
    uint16_t batVolt;         // battery voltage in tenths 151 is 15.1V
    uint16_t temp;           // temperature in tenths 322 is 32.2C
} pcm_status_t;
```

S T X	S T E Q P E	DSTMAC	L E N G	~ L E N G	PAYLOAD	C R C 1	C R C 2	E T X
2		0x14	3 byte dest MAC	8	~ 8 8 bytes (pcm_status_t)			3

TYPE – 0x14 bottom 4 bits are PCM_STATUS_PKT_TYPE(4) and top 4 are 0x10 PCM board

DSTMAC – broadcast (0xFFFFF) if PCM1 is the sender, otherwise PCM1 MAC

LENG - 0x08 (8 bytes)

~LENG - 0xF7

PAYLOAD – pcm_status_t structure

4.6 Switch Config Packet

There are basically two formats for Switch config packets, three-byte which are the SWITCH_APPLY, SWITCH_RESTORE, and SWITCH_GET packets and the SWITCH_UPDATE and SWITCH_SYNC packet where the payload includes the switch_config_t structure.

```
#define SWITCH_MAGIC 0xFEED
#define SWITCH_VERSION 2

// following are used for circuits and triggers
#define PRIMARY_ACTIVATION      0x01
#define SECONDARY_ACTIVATION    0x02
#define AUTO_TRIGGER_ACTIVATION 0x04
#define TRIGGER_ACTIVATION (PRIMARY_ACT|SECONDARY_ACT|AUTO_TRIGGER_ACT)

// commas for SWITCH_CONFIG_PKT_TYPE (if length < sizeof(switch_config_t)
typedef enum {
    SWITCH_APPLY = 0,
    SWITCH_RESTORE,
    SWITCH_UPDATE,
    SWITCH_GET,
    SWITCH_SYNC
} switch_config_command_t;

typedef struct {
    // bitmap of primary (circuits[0]) and secondary (circuits[1]) linked to a switch
    uint32_t circuits[2];
    // bitmap of primary (triggers[0]) and secondary (triggers[1]) qualifying a switch
    uint8_t triggers[2];
    uint32_t wig[2];      // unused
    uint32_t wag[2];      // unused
} switch_pcm_t;

typedef struct {
    uint16_t crc;          // CRC16 of remaining switch_config_t struct
    uint16_t magic;        // SWITCH_MAGIC
    uint16_t version;      // SWITCH_VERSION
    uint16_t sizeof_this;  // sizeof(switch_config_t)
    // don't add anything before this
    uint8_t config_index;  // 0..7 (auto trigger is 8)
    uint8_t switch_index;  // 0..7 (auto trigger is 8)
    uint8_t inputs;        // TOGGLE_INPUT .. DOUBLE_INPUT
    uint8_t unused;        // available for future use
    switch_pcm_t pcm[PCMS];
} switch_config_t;
```

4.6.1 Switch Config – SYNC or UPDATE Packet

The SYNC or UPDATE PAYLOAD is the switch_config_t structure and the in-memory config is updated without writing to the file system.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xA5	3 byte dest MAC			3 byte MAC (24 bits)	116 bytes (switch_config_t)			3

TYPE – 0xA5 bottom 4 bits are SWITCH_CONFIG_PKT_TYPE(5) and top 4 are 0x80 (SRCMAC) and 0x20 SWITCH

LENG - 0x77 (119 bytes = 3 + 116)

~LENG - 0x88

4.6.2 Switch Config – APPLY Packet

The APPLY PAYLOAD is 3 bytes and is set to SWITCH_APPLY(0). This will cause the device to write the in-memory pcm_config structure to the file named pcmN.bin. Where N is 0-3.

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB5	3 byte dest MAC			3 byte MAC (24 bits)	3 bytes (0, cfgx, swx)			3

TYPE – 0xB5 bottom 4 bits are SWITCH_CONFIG_PKT_TYPE(5) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x06 (4 bytes = 3 + 3)

~LENG - 0xF9

PAYLOAD – first byte is SWITCH_APPLY, second byte is config (0-7), 3rd byte is switch (0-7)

4.6.3. Switch Config – Restore Packet

The RESTORE PAYLOAD is 3 bytes and is set to SWITCH_RESTORE(1). This will cause the device to read the switchXY.bin file and store the contents in the in-memory switch_config structure.

S T X	S T E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB5	3 byte dest MAC			3 byte MAC (24 bits)	3 bytes (1, cfg, sw)			3

TYPE – 0xB5 bottom 4 bits are SWITCH_CONFIG_PKT_TYPE(5) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x06 (6 bytes = 3 + 3)

~LENG - 0xF9

PAYLOAD – first byte is SWITCH_RESTORE, second byte config (0-7) and 3rd is switch (0-7)

4.6.4 Switch Config – Get Packet

The GET PAYLOAD is 3 bytes and is set to SWITCH_GET(3). This will be sent to the PCM1 which will respond with a SWITCH_SYNC (switch_config payload) which causes the sending device to update the contents of it's corresponding in-memory switch_config structure.

S T X	S T E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2	E T X
2		0xB5	3 byte dest MAC			3 byte MAC (24 bits)	3 bytes (3, cfgx, swx)			3

TYPE – 0xB5 bottom 4 bits are SWITCH_CONFIG_PKT_TYPE(5) and top 4 are 0x80 (SRCMAC) and 0x30 TOUCHSCREEN

LENG - 0x06 (4 bytes = 3 + 3)

~LENG - 0xF9

PAYLOAD – first byte is SWITCH_GET(3), second byte is cfgx (0-7), 3rd byte is swx (0-7)

4.7 Switch Status Packet

The SWITCH_STATUS_PKT_TYPE(6) payload is the switch_status_t structure. This packet is sent when a switch is pressed or released by the mobile app, touchscreen or switch.

```
// following are in the switch status activation field defines
#define TOGGLE_INPUT          0x01
#define MOMENTARY_INPUT       0x02
#define LONG_INPUT            0x04
#define DOUBLE_INPUT          0x08
#define TURNOFF_INPUT         0x10 // turn off input
#define PRIMARY_INPUT (TOGGLE_INPUT|MOMENTARY_INPUT)
#define SECONDARY_INPUT (LONG_INPUT|DOUBLE_INPUT)

#define OFF_COLOR 0x000000
#define RED_COLOR 0xFF0000
#define GRN_COLOR 0x00FF00
#define BLU_COLOR 0x0000FF
#define FAULT_COLOR 0xFE0000

typedef struct {
    uint8_t index; // 0 to 63 (CONFIGS*SWITCHES-1)\
    uint8_t activation; // TOGGLE_INPUT .. DOUBLE_INPUT
    uint8_t owner; // rs485 address of controller that activated the switch
    uint8_t icon_id; // icon this switch is using, zero for none (unused)
    uint32_t color; // RGB color – OFF_COLOR .. FAULT_COLOR
} switch_status_t;
```

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	SRCMAC	PAYLOAD	C R C 1	C R C 2
2		0xB6	3 byte dest MAC			3 byte MAC (24 bits)	8 bytes (switch_status_t)		

TYPE – 0xB6 SWITCH_STATUS_PKT_TYPE(6) ORed with board type and SRCMAC (0x80)

DSTMAC – PCM1 MAC

PAYLOAD – switch_status_t structure

5.0 Circuit Status Packet

The CIRCUIT_STATUS_PKT_TYPE(0x0E) payload is the circuit_status_t structure. This packet is sent when a controller changes the circuit dim level, or when the circuit status has changed (OPEN, SHORT, FAULT or the current consumed has changed).

```
// following are in the circuit status field defines
#define CIRCUIT_OFF          0x00
#define CIRCUIT_NOM          0x01
#define CIRCUIT_FLASH        0x02
#define CIRCUIT_STROBE        0x03
#define CIRCUIT_BLINK         0x04
#define CIRCUIT_ONHOLD        0x08 // ignsense, temperature, etc. (output not enabled)
#define CIRCUIT_TICKS         0x10
#define CIRCUIT_FAULT         0x20
#define CIRCUIT_OPEN          0x40
#define CIRCUIT_SHORT         0x80
```

```
typedef struct {uint16_t hex; float_t dec;} adcVals_t;
typedef struct {
    uint16_t index; // 0 to 95 (CONFIGS*PCMS)\
    uint8_t status; // CIRCUIT_OFF .. CIRCUIT_SHORT
    uint8_t outCmd; // the PWM value (dim_level)
    uint8_t output; // the actual PWM value to write out
    uint8_t dim_level; // can adjust on the fly
    uint16_t blinkOn; // msec output should be on (5ms ticks)
    uint16_t blinkOff; // msec output show be off (5ms ticks)
    uint16_t period_ticks; // # of 5ms ticks in the period
    adcVals_t current; // circuit status in amps
} circuit_status_t;
```

S T X	S E Q	T Y P E	DSTMAC	L E N G	~ L E N G	PAYLOAD	C R C 1	C R C 2
2		0x1E	3 byte dest MAC			18 bytes (circuit_status_t)		

TYPE – 0x1E CIRCUIT_STATUS_PKT_TYPE(0xE) ORed with board type

DSTMAC – PCM1 MAC

PAYLOAD – circuit_status_t structure

6.0 sPOD Firmware Update (OTA)

Since PCM1 is the only sPOD device to provide a BLE interface, the firmware needs to implement two types of firmware updates: PCM1 update and all other sPOD devices.

The mobile application communicates directly with PCM1 over BLE when updating PCM1's firmware. PCM1 acts as a BLE proxy for non-PCM1 firmware updates. That is, it receives OTA packets from the mobile application and routes them to the correct non-PCM1 device over the RS485 bus.

Firmware for any sPOD device is a binary file with 16 bytes of version information embedded in the binary at offset 0x120 of the file. This is actually an ASCII string with the following format:

`"XXX-MM.mm.rr"`

Where **XXX** is either PDB, SDB, or TSB (corresponding to PCM, switch or touchscreen), **MM** is the major version number, **mm** is the minor version number and **rr** is the revision.

The OTA operation has been optimized, it currently takes around a minute to update an sPOD device's firmware. This was accomplished by sending 4Kbyte chunks of firmware image data at a time over BLE without waiting for an ACK. The data is sent as 32 130-byte packets, where the first 2 bytes are the offset into the 4K image data and the last 128 bytes are the data. The OTA downloader maintains a 4Kbyte image cache and a 32-bit image scoreboard. When a 130-byte packet is received, the bit corresponding to that packet's offset is set and the 128 bytes of data are copied into the 4Kb chunk. After all 32 packets have been transmitted, the mobile app sends a chunk complete packet with the crc32 of the 4K image. The PCM compares the received crc32 to the calculated one for the current 4K image chunk. If they match an "ACK" is returned and the host can send the next contiguous chunk of 4K data. If not, an "NAK" is returned along with the 32-bit scoreboard and a 64-byte array of the crc16 values for each of the 32 packets. The host can use this to retransmit the missing or incorrect packets.

There are two BLE characteristics to support the OTA algorithm, OTA Control and OTA Data.

The general OTA algorithm from the mobile app's perspective is:

1. Open the desired binary file containing the sPOD's firmware image.
2. Seek to offset 0x120 and read the 16-byte image descriptor from the file.
3. Send the OTA begin message described in section 6.1.
4. Wait for the OTA begin ACK message described in section 6.2.
5. Rewind to the beginning of the file.
6. Read 4K bytes from the file.
7. Write the first 128 bytes of the 4K chunk to PCM1 using the message described in section 6.3 and wait 10ms (this is to throttle the BLE traffic to avoid overruns).

8. Write the next 128 bytes of the 4K to PCM1 using the message described in section 6.3 (Wait 10ms after each packet write to avoid BLE overruns.)
9. Repeat step 8 for all 32 packets, (the offset increases by 0x80 (128) for each write which corresponds to each of the 32 packets – offsets will be 0x000, 0x080, 0x100 .. 0xF80).
10. Write the 4K chunk complete message to PCM1 using the message described in section 6.4.
11. Wait for the chunk complete ACK message described in section 6.5.
12. If the response is a NAK, resend any 128-byte packet that was not received correctly.
Note: if “NAK-LENG” or “NAK-WRITE” are returned the OTA should terminate with a failure. Otherwise “NAK-SCORE” and “NAK-CRC” can be treated the same by re-sending the 128-byte packet if either the ScoreBoard bit for that packet is not set or the CRC16[n] for that packet does not match.
13. Repeat steps 9-11 until an ACK response is received.
14. If this is the first 4K chunk complete message (ie. writing out the first 4K bytes of the firmware image data) wait 3 seconds for the sPOD device to erase the flash where this firmware image data will be written.
15. If the entire firmware binary image file has been sent, signal the OTA is complete
16. Otherwise, read the next 4K bytes from the binary image file and go to step 7.

6.1 Mobile APP OTA Control Packet (begin OTA update)

The mobile app sends this packet to PCM1 using the OTA Control characteristic when it wants to start a firmware update.

Bytes 1-3	Byte 4	Bytes 5-20	Bytes 21-24
MAC	Force	Image Desc	Image Length

Where:

- MAC – is the least significant 24-bits of the sPOD device's BLE MAC address
- Force – set to 0xF0 to force an update regardless of the currently running version, 0x00 will only update if the new version is more recent than the currently running version
- ImageDesc – the mobile app reads 16 bytes of the firmware binary image file at offset 0x120 and deposits into ImageDesc
- ImageLength – this is the size of the firmware binary image file specified in bytes

6.2 PCM1 OTA Control Packet ACK/NAK (begin OTA update)

PCM1 responds to the mobile app's packet using OTA Control characteristic with the following:

Bytes 1-3	Byte 4	Bytes 5	Bytes 6-N
MAC	Length	0x80	Message

Where:

- MAC – is the least significant 24-bits of the sPOD device's BLE MAC address
- Length – length of the ASCII Message
- 0x80 – denotes this is a response to the starting control packet (6.1) sent by mobile app
- Message – the ACK or NAK, messages are as follows:
 - “OK:UPDATING” – this version is more recent than what is currently running, updating...
 - “OK:FORCE UPDATE” – 0xF0 force option was specified by mobile app, updating...
 - “ER:INCOMPATIBLE” – firmware image was built for different type of sPOD device, fail
 - “ER:TOO OLD” – this version is older than what is currently running, fail
 - “ER:ALREADY MATCH” – this version is currently running, fail

6.3 Mobile APP OTA Data Packet

Generally, the mobile app reads the next 4K bytes from the firmware binary image file and then sends 32 128-byte data packets where the offset is increased by 128 for each packet. The mobile app sends to PCM1 using the OTA Data characteristic when it wants to update the next 128-byte chunk of firmware. Following is the format of one of those packets:

Bytes 1-3	Bytes 4-5	Bytes 6-133
MAC	Offset	Image Data

Where:

- MAC – is the least significant 24-bits of the sPOD device's BLE MAC address
- Offset –the offset of this 128-byte data packet into the 4Kb image read from the firmware binary image file
- ImageData – the next 128-bytes of data read from the firmware binary image file

The mobile app reads the next 4Kbytes of the firmware binary image file and then writes 32 of these packets to PCM1 starting with offset 0x000, 0x080, 0x100 ... 0xF80.

Note: in the test application it was necessary to wait 10ms between packet writes to avoid buffer overflow. Also note, the OTA control characteristic write was retried at most 5 times with an exponential backoff between each write to drain BLE output buffers. The millisecond delay used for the backoff was (retry*retry*50). So first retry delay was 50ms, second retry delay was 200ms, third delay was 450ms, and final retry delay was 800ms.

6.4 Mobile APP OTA Control Packet (4Kb chunk complete)

After the mobile app has completed sending 32 128-byte packets using the packet format described in section 6.3 previously, it sends a 4Kb chunk complete message to PCM1 using the OTA Control characteristic.

Bytes 1-3	Byte 4	Byte 5	Bytes 6-9	Bytes 10-13
MAC	Length (10)	0x50	Image CRC	CRC

Where:

- MAC – is the least significant 24-bits of the sPOD device's BLE MAC address
- Length – the length is 10 (it includes the length byte)
- 0x50 – this marks this packet as a CHUNK complete
- ImageCRC – the 32-bit CRC in little endian format of the 4Kb image chunk
- CRC – the 32-bit CRC in little endian format of the first 9 bytes of the packet

The mobile app waits for a response from PCM1 to determine if it can move on to sending the next 4Kb chunk or re-send any 128-byte data packets (note, the 4Kb image chunk is retained until all 32 128-byte data packets have been received correctly). *If this is the first 4Kb chunk sent to PCM1, the mobile app should **wait 3 seconds** after it gets a response back from PCM1 to allow time for PCM to erase the flash in preparation for the new firmware image.*

We could erase the flash on the fly as new sections are downloaded, but it is actually faster to bulk erase a section of flash so we pre-erase all of the flash once a new download has started.

6.5 PCM1 OTA Control Packet (4Kb chunk complete ACK)

When PCM1 receives the 4k chunk complete message from the mobile app, it sends the following packet to the mobile app using the OTA Control characteristic:

Bytes 1-3	Byte 4	Byte 5	Bytes 6-21	Bytes 22-25	Bytes 26-89	Bytes 90-93
MAC	Length (90)	0x50	Message	Score Board	CRC16[32]	CRC32

Where:

- MAC – is the least significant 24-bits of the sPOD device's BLE MAC address
- Length – the length is 90 (it includes the length byte)
- 0x50 – this marks this packet as a CHUNK complete ACK
- Message – the ACK/NAK message with the following formats:
 - ACK** – all good, move on to next 4Kb chunk
 - NAK-LENG** – the mobile app chunk complete packet length was not 10 or PCM1 does not think there is an OTA running currently
 - NAK-SCORE** – the ScoreBoard does not include all 32 128-byte data packets
 - NAK-CRC** – the CRC32 of the 4kb chunk does not match the ImageCRC provided in the mobile app's 4k chunk packet
 - NAK-WRITE** – unable to write the 4Kb chunk to flash
- CRC16[32] – 32 16-bit CRCs in little endian format corresponding to the 32 128-byte packets comprising the 4Kb chunk
- CRC32 – the 32-bit CRC in little endian format of the first 9 bytes of the packet

If ACK was returned and this is the last 4Kb chunk, the OTA is complete!

7.0 Mobile App Firmware Update Display

As described in section 4.6 BOARD_INFO_PAYLOAD, the Mobile App will receive BOARD_INFO packets over the BLE COMM characteristic. The Mobile App should save this data and display it to the operator when the firmware update display page is selected. The display should include the following information:

BOARD	MAC	VERSION	AVAILABLE
PCM-0	c3:89:9c	v1.2.38	v1.3.7
PCM-1	c3:89:aa	v1.1.4	v1.3.7
SWITCH-4	c3:88:12	v3.0.6	v4.0.8
SWITCH-6	c3:55:fe	v2.0.4	v4.0.8
TOUCHSCREEN-5	c3:77:34	v2.2.4	v2.2.4

The operator can select one of these lines and hit a key to initiate a firmware update. The mobile app should confirm that the operator really wants to do a firmware update (possibly with a warning that the system will not be functional during the update and only one update is allowed at a time). If they choose to do the update, a progress bar should be displayed and the firmware should be updated along with error messages displayed if the update fails. Refer to section 6.0 for a list of possible update errors.

Note that the currently running version is provided by the BOARD_INFO message with the major/minor/revision fields and the available version is obtained by polling the cloud for new firmware releases or saved on the mobile app device's file system. The firmware images are named something like:

```
spod_pcm.v1_3_7.bin  
spod_switch.v4_0_8.bin  
spod_touchscreen.v2_2_8.bin
```

Or however you want to name them, as long as they include the version number in the filename and it matches the ImageDesc at offset 0x120 in the file. Example image descriptors are:

```
"PDB-01.02.38" – PCM image with version v1.2.38  
"SDB-03-00-06" – SWITCH image with version v3.0.6  
"TSB-02-02-04" – TOUCHSCREEN image with version v2.2.4
```

The board info packet payload (section 4.6) is an array of 16 elements with the following structure:

```
typedef struct {
    uint8_t mac[3];
    uint8_t board_type;
    uint8_t addr;
    uint8_t major;
    uint8_t minor;
    uint8_t revision;
} board_info_t;
```

In the C programming language, it is defined as:

```
board_info_t board[16];
```

The fields in the table list above are constructed with the following pseudo code:

- **BOARD** – `snprintf(board_name, sizeof(board_name), "%s-%d", <board-type>, board[n].addr)` where <board-type> string is:
If `board[n].board_type` is 0x00 → "unspecified"
If `board[n].board_type` is 0x10 → "PCM"
If `board[n].board_type` is 0x20 → "SWITCH"
If `board[n].board_type` is 0x30 → "TOUCHSCREEN"
- **MAC** – `snprintf(mac_string, sizeof(mac_string), "%02x:%02x:%02x", board[n].mac[0], board[n].mac[1], board[n].mac[2])`
- **VERSION** – `snprintf(version, sizeof(version), "v%d.%d.%d", board[n].major, board[n].minor, board[n].revision)`
- **AVAILABLE** – constructed from the filename of the most recent image for that particular board type